

COS - AT1 - Pseudocode writing tasks

1) task scheduling system using directed graph

Problem :-

Design a task scheduling system for a Project where some tasks depends on the completion of other tasks

Graph model

- Represent each task as a vertex (node)
- Represent a dependency as a directed edge
- If task A must be completed before task B, create an edge $A \rightarrow B$.
- A valid schedule can be generated using Topological

Pseudo code :

Algorithm Taskscheduling (Graph G)

Input :

Directed graph $G = (V, E)$

V = set of tasks

E = set of task dependencies

Output :

validation execution order of tasks.

Begin

For Each task v in V DO

$\text{indegree}[v] \leftarrow 0$

End For

For Each edge (u, v) in E DO

$\text{indegree}[v] \rightarrow \text{indegree}[v] + 1$

END For

create an empty queue Q

For Each task v in V DO

 if $\text{indegree}[v] = 0$ then

 Enqueue(Q, v)

 End if

End For

create an empty list $order$

while Q is not empty DO

$u \leftarrow \text{Dequeue}$

 ADD u to $order$

$\text{in degree}[v] \leftarrow \text{indegree}[v]$

 IF $\text{indegree}[v] = 0$ then

 End if

 End For

End while

If number of tasks in $order < |V|$
then Print "cyclic dependency detected"

 Print "valid schedule cannot be generated"

Else

 Print "valid Task Execution order"

 Print $order$

DFS on a Graph using Adjacency List.

Problem:

perform depth first DFS on graph represented using an adjacency list

Pseudo code:

Algorithm DFS(u)

Begin

visited [u] $\leftarrow$ True

Print u

For each vertex v in adj [u] DO

IF visited [v] = False then

DFS(v)

End if

End For

END

Cycle Detection

0 $\rightarrow$ not visited

1 $\rightarrow$ currently being visited

2 $\rightarrow$ completely visited

Algorithm detect cycle (v)

Begin

state [v] $\leftarrow$ 1

For each v in adj[v] DO

if state [v] = 1 then

Return true

End if

If state [v] = 0 AND

Detect cycle (v) = true then

Return true

End if

End for

state [v] $\leftarrow$ 2

Return false

END.

Time complexity

• DFS time complexity : $O(V+E)$

DFS space complexity : $O(V)$

Ans
31/10/20